Spam Email Classification

Project Title

NLP-Based Spam Email Classification Using TF-IDF and Multinomial Naive Bayes

Self-Study Project – Natural Language Processing

Programming Language: Python
Platform: Google Colab
NLP Technique: TF-IDF
Machine Learning Algorithm: Multinomial Naive Bayes
Problem Type: Text Classification

1. Problem Identification

Spam emails are unwanted messages that may contain advertisements, fraudulent content, malicious links, or irrelevant information.

The objective of this project is to develop an NLP-based machine learning system that automatically classifies email messages into two categories:

- Spam
- Ham (Not Spam)

The system analyzes the textual content of an email and predicts whether the message is spam or legitimate.

2. Problem Relevance

Spam email is a common problem for email users and organizations. Manually identifying unwanted emails is time-consuming and can result in important messages being missed.

Natural Language Processing is suitable for this problem because emails contain textual information. NLP techniques can convert the text into numerical features that can be processed by machine learning algorithms.

The proposed system can help:

- Email users
- Organizations
- Email service providers
- Automated email filtering systems

The system can potentially be integrated into an email filtering application to automatically identify unwanted messages.

3. Objectives

The main objectives of this project are:

1. To understand the problem of spam email classification.
2. To obtain a suitable spam and ham text dataset from Kaggle.
3. To preprocess the email text using NLP techniques.
4. To convert text into numerical features using TF-IDF.
5. To train a Multinomial Naive Bayes classification model.
6. To classify emails as spam or ham.
7. To evaluate the performance using Accuracy, Precision, Recall, F1-score, and Confusion Matrix.
8. To analyze the limitations and possible improvements of the system.

What you need

You need:

1. **Google Colab**
2. **Kaggle account**
3. A suitable **Spam/Ham text dataset from Kaggle**
4. Python
5. Libraries:
    - pandas
    - numpy
    - matplotlib
    - seaborn
    - scikit-learn
    - nltk

We will use **Multinomial Naive Bayes + TF-IDF**

Dataset:

```
1 df.head()
```

	label	text	clean_text
0	1	ounce feather bowl hummingbird opec moment ala...	ounce feather bowl hummingbird opec moment ala...
1	1	wulvob get your medircations online qnb ikud v...	wulvob get your medircations online qnb ikud v...
2	0	computer connection from cnn com wednesday es...	computer connection from cnn com wednesday esc...
3	1	university degree obtain a prosperous future m...	university degree obtain a prosperous future m...
4	0	thanks for all your answers guys i know i shou...	thanks for all your answers guys i know i shou...

Next steps: Generate code with df | New interactive sheet

Dataset information:

```
1 df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 83448 entries, 0 to 83447
Data columns (total 3 columns):
 #   Column      Non-Null Count  Dtype 
---  ------      --------------  ----- 
 0   label       83448 non-null  int64 
 1   text        83448 non-null  object
 2   clean_text  83448 non-null  object
dtypes: int64(1), object(2)
memory usage: 1.9+ MB
```

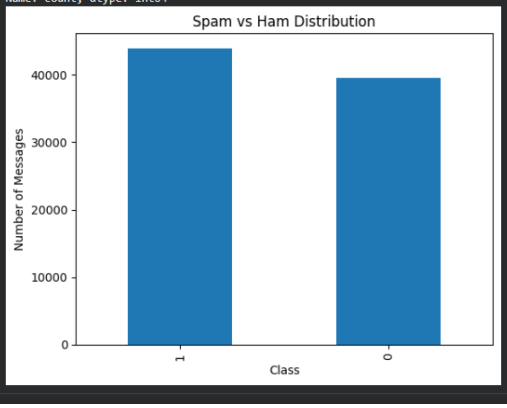
Class distribution

Check the class distribution

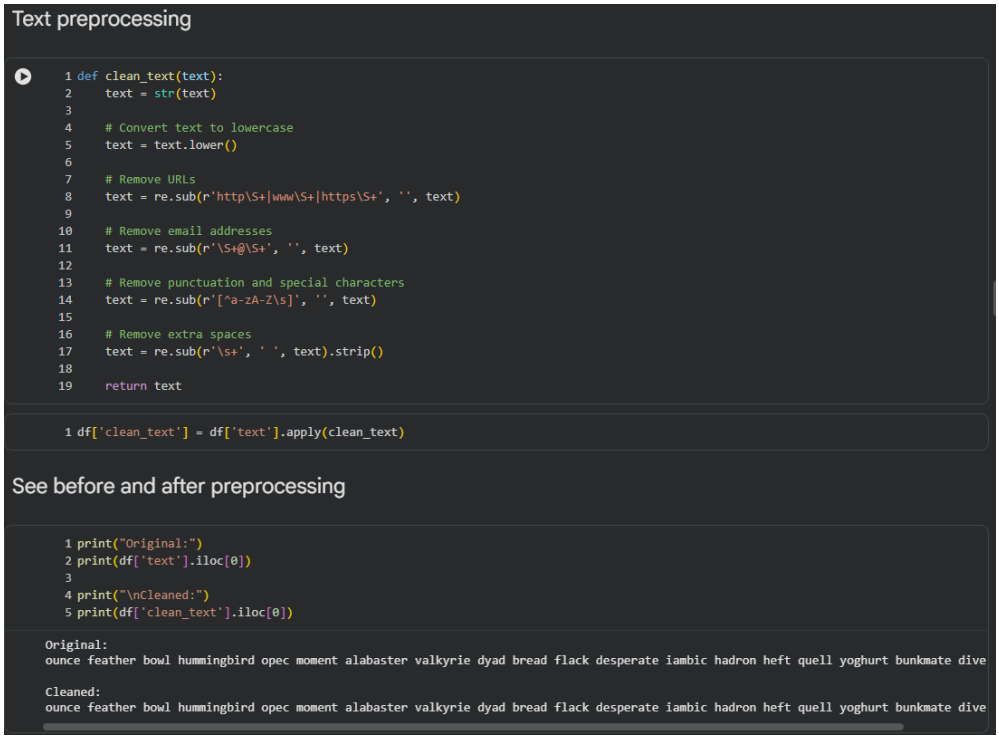
This tells us whether the dataset is balanced or imbalanced.

```
1 print(df['label'].value_counts())
2 df['label'].value_counts().plot(kind='bar')
3
4 plt.title('Spam vs Ham Distribution')
5 plt.xlabel('Class')
6 plt.ylabel('Number of Messages')
7 plt.show()
```

```
label
1    43910
0    39538
Name: count, dtype: int64
```



Preprocessing before/after



TF-IDF shape

What is TF-IDF?

You should understand this for your viva.

TF-IDF = Term Frequency–Inverse Document Frequency

It converts text into numerical values.

For example:

"Congratulations you won a prize"

becomes numerical features.

TF-IDF gives greater importance to words that are useful for distinguishing documents.

TF-IDF Vectorization

25. What is TF-IDF?

You should understand this for your viva.

TF-IDF = Term Frequency–Inverse Document Frequency

It converts text into numerical values.

For example:

"Congratulations you won a prize"

becomes numerical features.

TF-IDF gives greater importance to words that are useful for distinguishing documents.

```
tfidf = TfidfVectorizer(
    max_features=5000,
    stop_words='english'
)
```

```
#Train the vectorizer:
X_train_tfidf = tfidf.fit_transform(X_train)
#Transform the test data:
X_test_tfidf = tfidf.transform(X_test)
print("Training TF-IDF shape:", X_train_tfidf.shape)
print("Testing TF-IDF shape:", X_test_tfidf.shape)
```

```
Training TF-IDF shape: (66758, 5000)
Testing TF-IDF shape: (16690, 5000)
```

Classification Report

Classification Report

```
print(classification_report(y_test, y_pred))
```

```
              precision    recall  f1-score   support

           0       0.95      0.97      0.96      7908
           1       0.97      0.96      0.97      8782

    accuracy                           0.96     16690
   macro avg       0.96      0.96      0.96     16690
weighted avg       0.96      0.96      0.96     16690
```

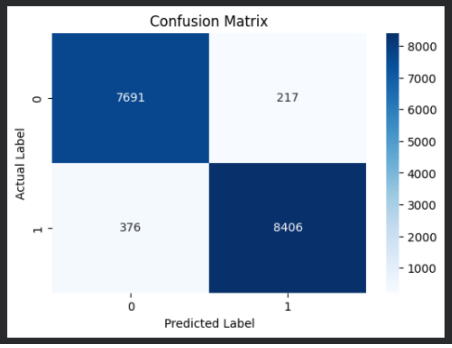
Confusion matrix

Confusion Matrix

```
cm = confusion_matrix(y_test, y_pred)

print(cm)
```

```
[[7691  217]
 [ 376 8406]]
```

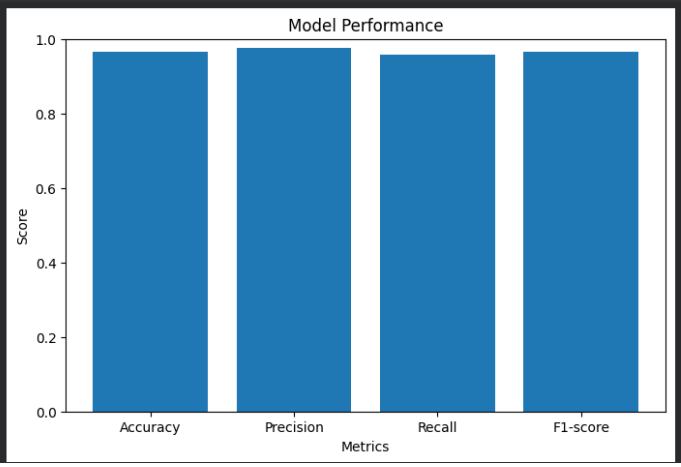
```
plt.figure(figsize=(6, 4))

sns.heatmap(
    cm,
    annot=True,
    fmt='d',
    cmap='Blues'
)

plt.title('Confusion Matrix')
plt.xlabel('Predicted Label')
plt.ylabel('Actual Label')
plt.show()
```



Performance graph

Results graph

```
metrics = ['Accuracy', 'Precision', 'Recall', 'F1-score']
values = [accuracy, precision, recall, f1]

plt.figure(figsize=(8, 5))

plt.bar(metrics, values)

plt.title('Model Performance')
plt.xlabel('Metrics')
plt.ylabel('Score')
plt.ylim(0, 1)

plt.show()
```



Sample predictions

Sample spam/ham predictions

```
sample_messages = [
    "Congratulations! You have won a free prize. Click here to claim now.",
    "Hi, can we meet tomorrow to discuss the project?",
    "You have been selected for a cash reward. Claim your money now!",
    "Please send me the assignment document before tomorrow."
]
```

```
#Clean them:
clean_samples = [clean_text(message) for message in sample_messages]
#Convert to TF-IDF:
sample_tfidf = tfidf.transform(clean_samples)
#Predict:
predictions = model.predict(sample_tfidf)
for message, prediction in zip(sample_messages, predictions):
    if prediction == 1:
        result = "SPAM"
    else:
        result = "HAM"

    print("Message:", message)
    print("Prediction:", result)
    print("-" * 60)
```

```
Message: Congratulations! You have won a free prize. Click here to claim now.
Prediction: SPAM
------------------------------------------------------------
Message: Hi, can we meet tomorrow to discuss the project?
Prediction: HAM
------------------------------------------------------------
Message: You have been selected for a cash reward. Claim your money now!
Prediction: SPAM
------------------------------------------------------------
Message: Please send me the assignment document before tomorrow.
Prediction: HAM
```

Limitations

The project has some limitations:

1. The model depends on the quality and diversity of the dataset.
2. New types of spam emails may not always be classified correctly.
3. The system mainly relies on textual information.
4. Images and other non-textual content inside emails are not analyzed.
5. Spam messages containing unusual or previously unseen vocabulary may be difficult to classify.
6. The model may perform differently on datasets from different domains.

Future Scope

The project can be improved in the future by:

1. Using a larger and more diverse email dataset.
2. Applying more advanced NLP preprocessing techniques.
3. Comparing Naive Bayes with Logistic Regression, SVM, and Random Forest.
4. Exploring deep learning and Transformer-based models.
5. Supporting multiple languages.
6. Developing a real-time spam detection system.
7. Deploying the model as a web application.
8. Integrating the system with an email service.

Conclusion

This project developed an NLP-based spam email classification system using TF-IDF and Multinomial Naive Bayes.

The email text was first cleaned and pre-processed. TF-IDF was then used to convert the textual data into numerical features. A Multinomial Naive Bayes classifier was trained using these features to classify messages as spam or ham.

The model was evaluated using Accuracy, Precision, Recall, F1-score, and Confusion Matrix.

This project helped demonstrate how Natural Language Processing and Machine Learning can be combined to solve a practical text classification problem.